CS 214: Data Structures
Stacks and Array-based Implementation

Slide contents follow

Kruse and Leung "Data Structures & Program Design in C"

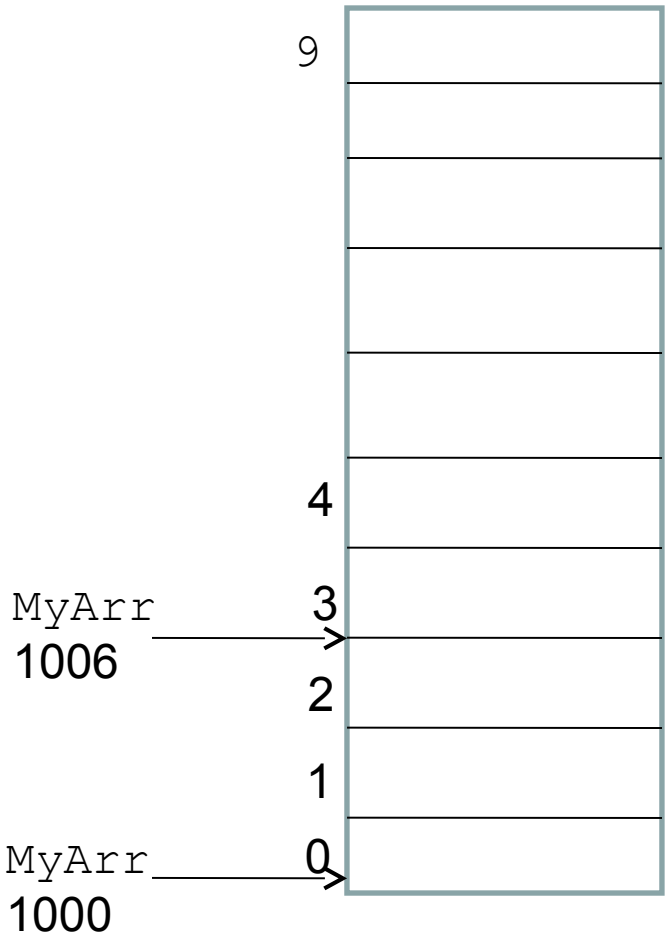
Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

Introduction: array as a familiar data structure

Implementation level



User Level

```
int MyArr[10]; //Many things
happen here.
```

1- reserving a contiguous space in memory, so that:

$$\text{memory size} = \text{element size} \times \#\text{elements}$$

2- giving the starting address the name `MyArr`

```
MyArr[3] = 27;   //Many things
happen here.
```

1- calculates the location address:

$$\begin{aligned}\text{Loc address} &= \text{MyArr} + 3 * \text{sizeof(int)} \\ &= \text{MyArr} + 6\end{aligned}$$

2- Stores 27 in that location.

© Waleed A. Yousef 2008

Information hiding (Encapsulation):

- The use of functions. You use the structure at the "User Level" without caring about the details at the "Implementation Level".

- Your program, i.e., the user level, does not change even if the implementation of the used structure is changed.

- Your program is clear from the logical point of view.

- Enhancing the Top-Down-Design.

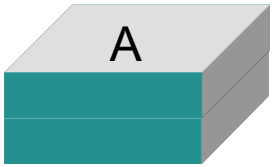
© Waleed A. Yousef 2008

Motivation: What and Why `Stacks`?

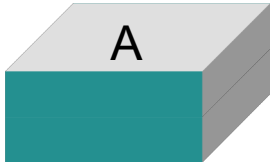
Push box Q onto empty stack:



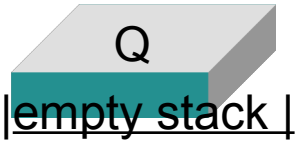
Push box A onto stack:



Pop a box from stack:



Pop a box from stack:

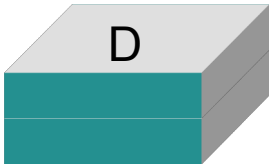


© Waleed A. Yousef 2008

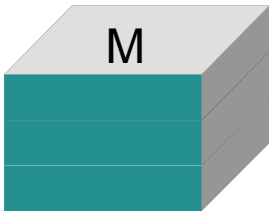
Push box R onto stack:



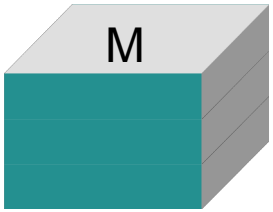
Push box D onto stack:



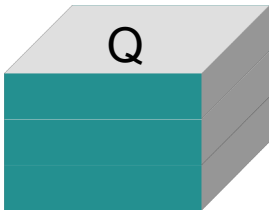
Push box M onto stack:



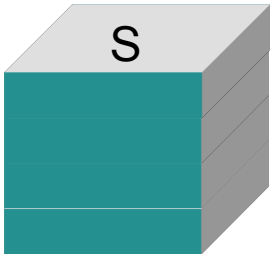
Pop a box from stack:



Push box Q onto stack:



Push box S onto stack:



© Waleed A. Yousef 2008

Motivation for Stacks as new Data Structures

Assume that we need to read a line of text and write it back in a reverse order. In general, store some data and retrieve it in a reverse order.

```
void ReverseRead(void){
  StackEntry  item;//StackEntry should be defined as char
  Stack stack;
  CreateStack(&stack);//Initialize the stack to be empty

  while (!StackFull(&stack) && (item = getchar()) != '\n')
    Push(item, &stack);//Push each item onto the stack

  while (!StackEmpty(&stack)){
    Pop(&item, &stack); //Pop an item from the stack.
    putchar(item);
  }
  putchar('\n');
}
```

© Waleed A. Yousef 2008

Again: Information hiding (Encapsulation):

- The use of functions. You use the structure at the "User Level" without caring about the details at the "Implementation Level".

- Your program, i.e., the user level, does not change even the implementation of the used structure is changed.

- Your program is clear from the logical point of view.

- Enhancing the Top-Down-Design.

© Waleed A. Yousef 2008

Definitions, where every thing should start from!

Definition: ***type*** is a set of values and a set of operations on those values.

Example: We can define a datatype `boolean` that takes the set of values `{0, 1}` together with the operations `AND`, `OR`, and `NOT`.

Example: `int`, in `C`, is the set consisting all of the integers between `INT_MIN` $(-(2^{15} - 1))$ and `INT_MAX` $(2^{15} - 1)$, which are defined in the header file `limits.h`

Definition: A ***Sequence of length 0*** is empty. A ***sequence of length*** $n \geq 1$ of elements from a set `T` is an ordered pair (S_{n-1}, t) where S_{n-1} is a sequence of length $n-1$ of elements from `T`, and `t` is an element of `T`.

© Waleed A. Yousef 2008

Definition: ***Abstract Data Type (ADT)*** is a data type that is accessed only through an **interface** (or **Accessing mechanism**). We refer to a program that uses an ADT as a **client** (or **user**) and a program that specifies the data type as an **implementation**

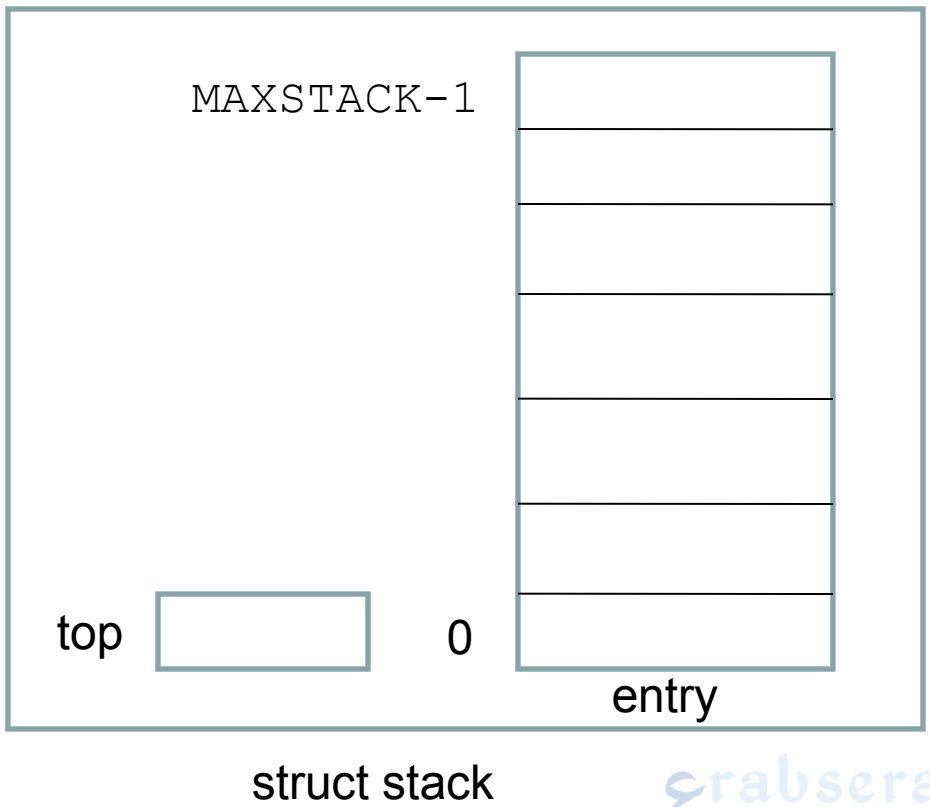
Definition: ***Stack*** of elements of type *T is a finite sequence of elements of T together* with the following operations:

1. Create the stack, leaving it empty.
2. Determine whether the stack is empty or not.
3. Determine whether the stack is full or not.
4. Find the size of the stack.
5. Push a new entry onto the top of the stack, provided the stack is not full.
6. Pop the entry off the top of the stack, provided the stack is not empty.
7. Retrieve the Top entry off the stack, provided the stack is not empty.
8. Traverse the stack, visiting each entry.
9. Clear the stack to make it empty.

© Waleed A. Yousef 2008

Type Definition

```
struct stack{
    int top;
    StackEntry entry[MAXSTACK];
}
```



`StackEntry` and `MAXSTACK` should be defined in the User Level.

For the moment forget about them. We will mention later how they should be defined.

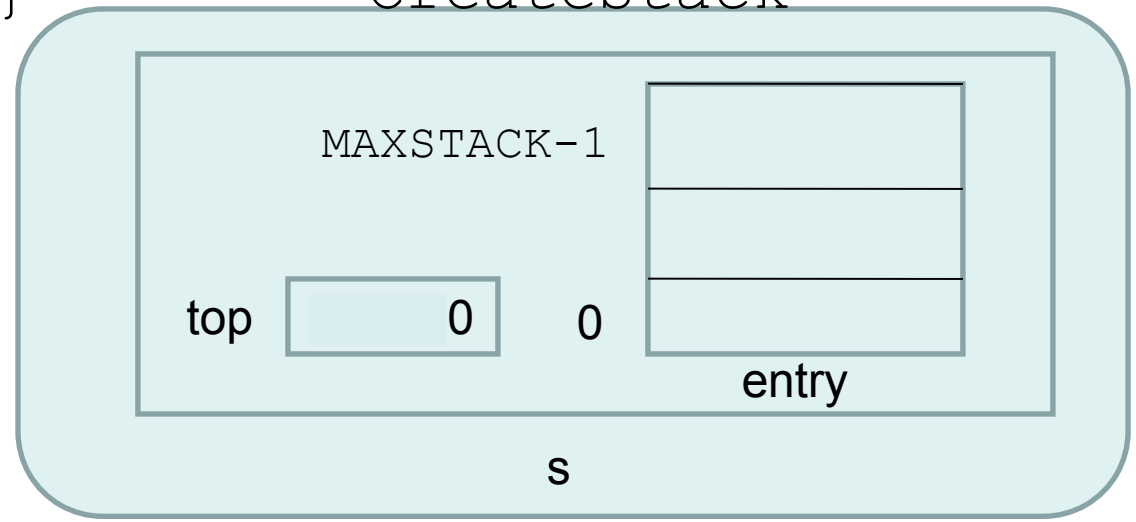
© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
void CreateStack(Stack s){
    s.top=0;
}
```

CreateStack



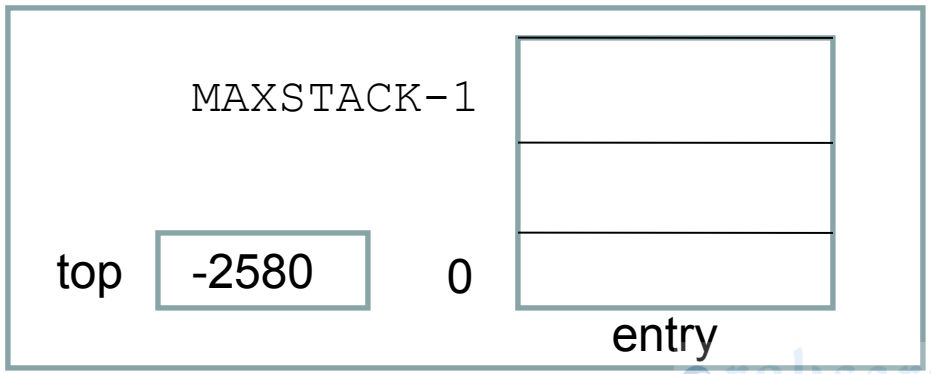

```
void main(){

Stack s;

CreateStack(s);


}
```

When we return:



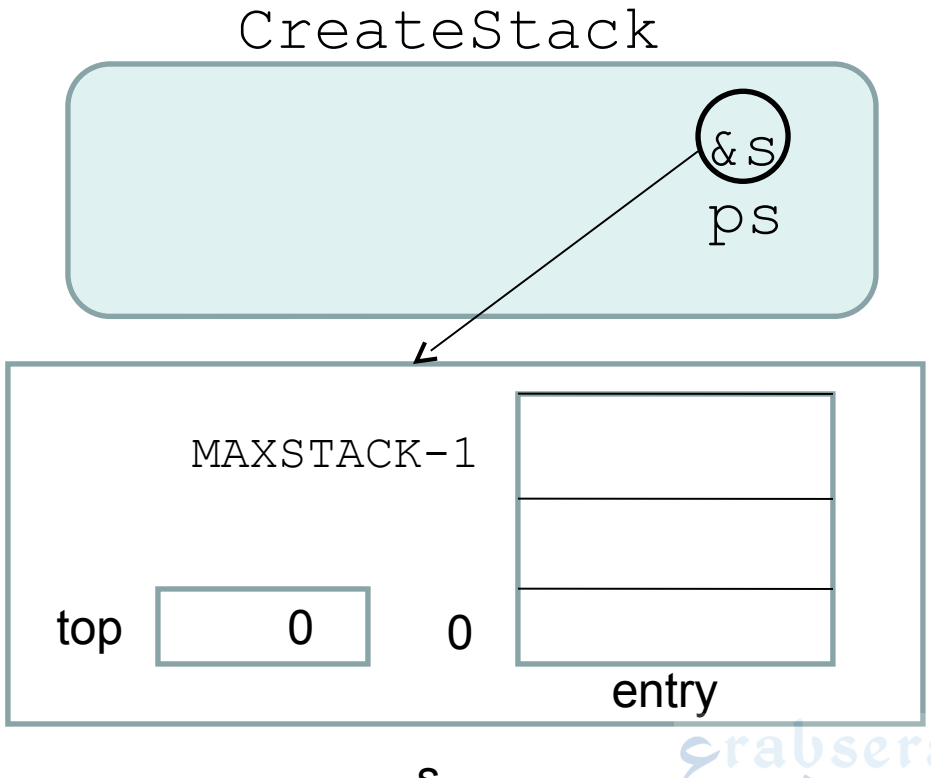
© Waleed A. Yousef 2008

Implementation level (what really happens)

But this way

User Level (interface)

```
void CreateStack(Stack *ps){
    ps->top=0;
}
```




```
void main(){

Stack s;

CreateStack(&s);


}
```

When we return:

The execution time does not depend on n; therefore the complexity is: $\Theta(1)$

`top` is the index of the first available place.

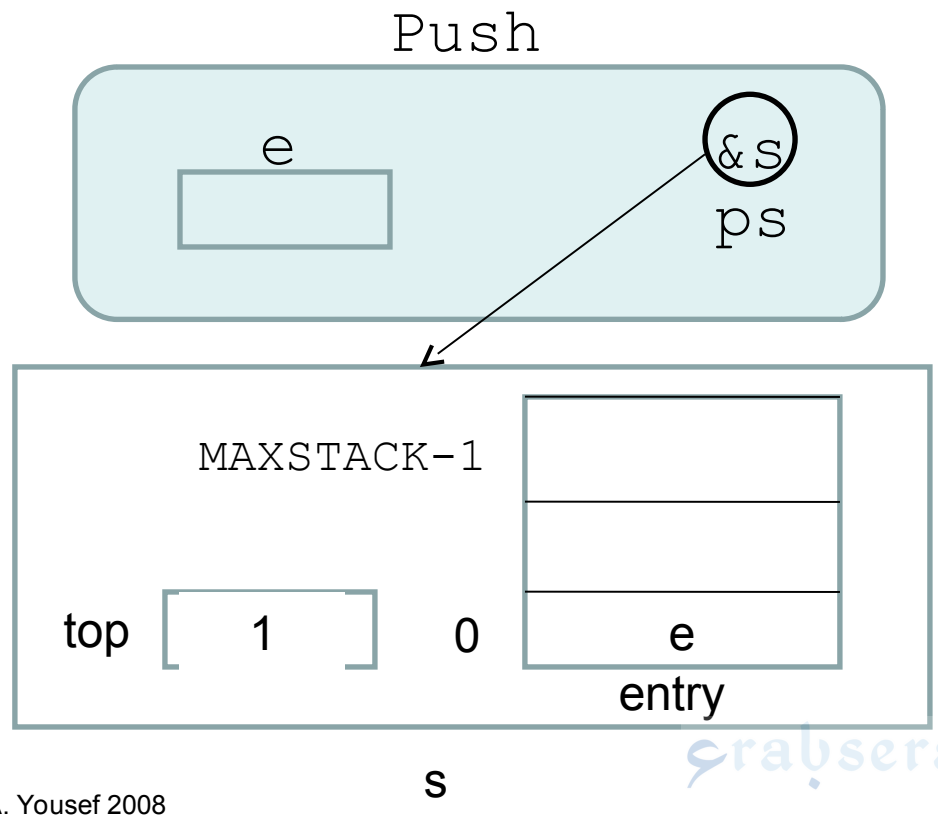
© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
void Push(StackEntry e, Stack *ps){
    ps->entry[ps->top]=e;
    ps->top++;
}
```

ps->entry[ps->top++]=e;




```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
Push(e, &s);

}
```

© Waleed A. Yousef 2008

```
/*Pre: The stack is initialized and not full
  Post: The element e has been stored at the top of
the stack; and e does not change*/
void Push(StackEntry e, Stack *ps){
     ps->entry[ps->top++]=e;
}
```

The user has to check before calling `Push`

Other ways (no precondition) are:

```
if (ps->top==MAXSTACK)
     printf("Stack is full");
else ps->entry[ps->top++]=e;
//but this is not professional
```

```
int Push(…){
   if (ps->top==MAXSTACK)
       return 0;
   else {
       ps->entry[ps->top++]=e;
       return 1;
   }//This is fine
```

```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
if (!StackFull(s))
      Push(e, &s);
}

if (!Push(e, &s))
      …
```

© Waleed A. Yousef 2008

Implementation level (what really happens)

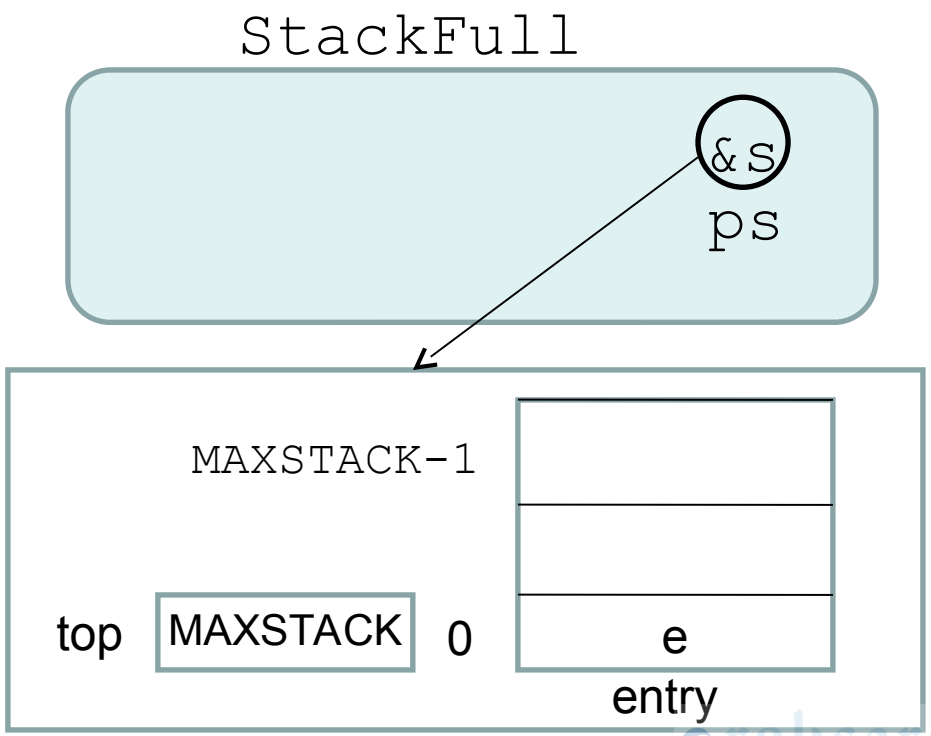
```
int StackFull(Stack *ps){
    if (ps->top==MAXSTACK)
       return 1;
    else
       return 0;
}
```

User Level (interface)

```
return ps->top >= MAXSTACK;
```

```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
if (!StackFull(&s))
      Push(e, &s);
}
```

It could be: `StackFull(s)` but this wastes memory and time of copying.



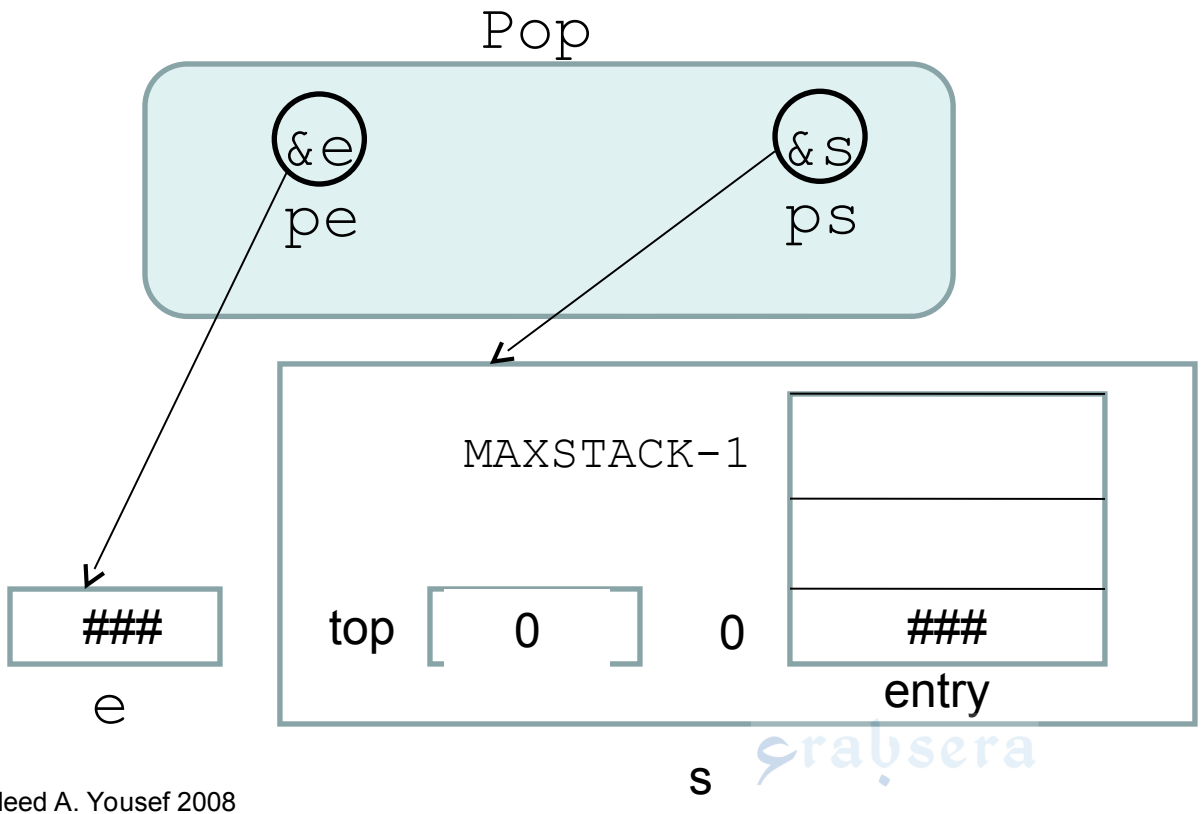
© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
void Pop(StackEntry *pe, Stack *ps){
   ps->top--;
   *pe=ps->entry[ps->top];
}
```

```
*pe=ps->entry[--ps->top];
```




```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
Pop(&e, &s);

}
```

© Waleed A. Yousef 2008

```
/*Pre: The stack is initialized and not empty
  Post: The last element entered is returned*/

void Pop(StackEntry *pe, Stack *ps){
      *pe=ps->entry[--ps->top];
}
```

The user has to check before calling `Pop`

Other ways (no precondition) are:

```
if (ps->top==0)
      printf("Stack is Empty");
else *pe=ps->entry[--ps->top];
//but this is not professional

int Pop(…){
   if (ps->top==0)
       return 0;
   else {
       *pe=ps->entry[--ps->top];
       return 1;
   }//This is fine
```

```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
if (!StackEmpty(s))
      Pop(&e, &s);
}

if (!Pop(&e, &s))
      …
```

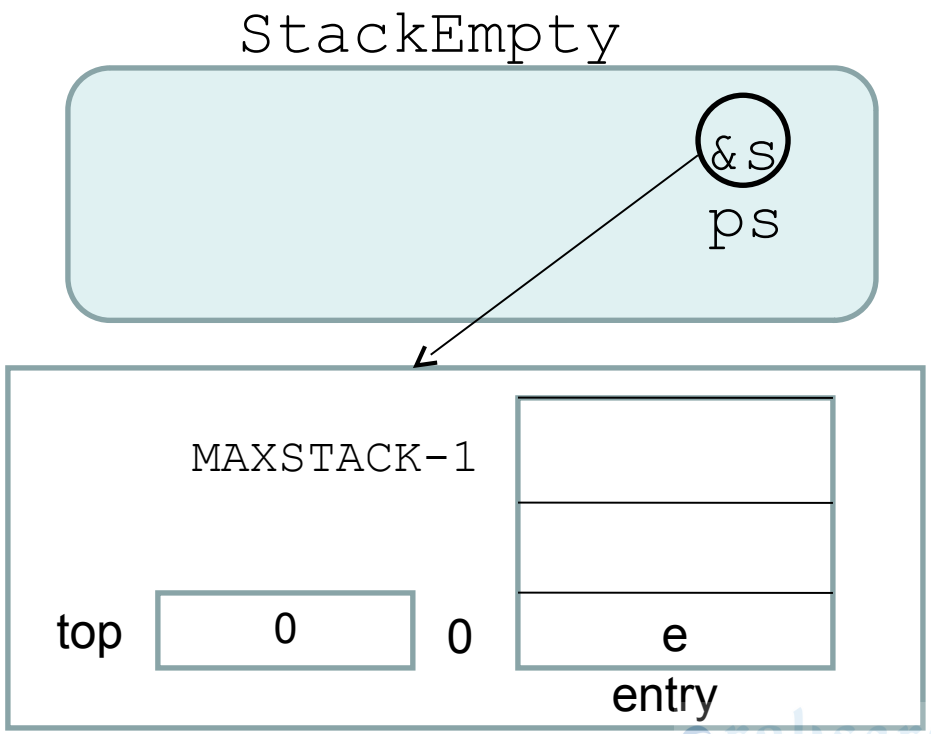
© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
int StackEmpty(Stack *ps){
    if (ps->top==0)
       return 1;
    else
       return 0;
}
```

```
return !ps->top;
```




```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
if (!StackEmpty(&s))
      Pop(&e, &s);
}
```

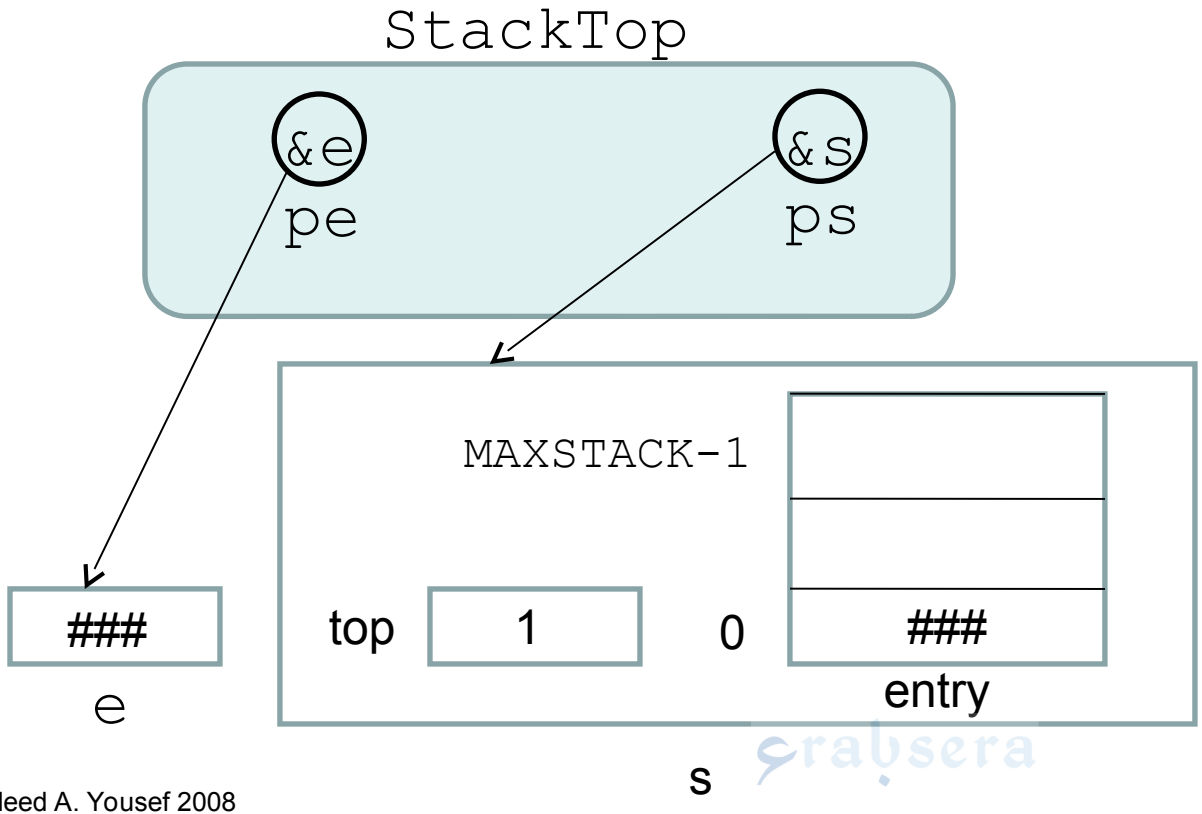
It could be: `StackEmpty(s)` but this wastes memory and time of copying.

© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
//Same preconditions of Pop.
void StackTop(StackEntry *pe, Stack *ps){
     *pe=ps->entry[ps->top-1];
}
```




```
void main(){
StackEntry e;
Stack s;
     :
CreateStack(&s);
     :
StackTop(&e, &s);
}
```

It could be:

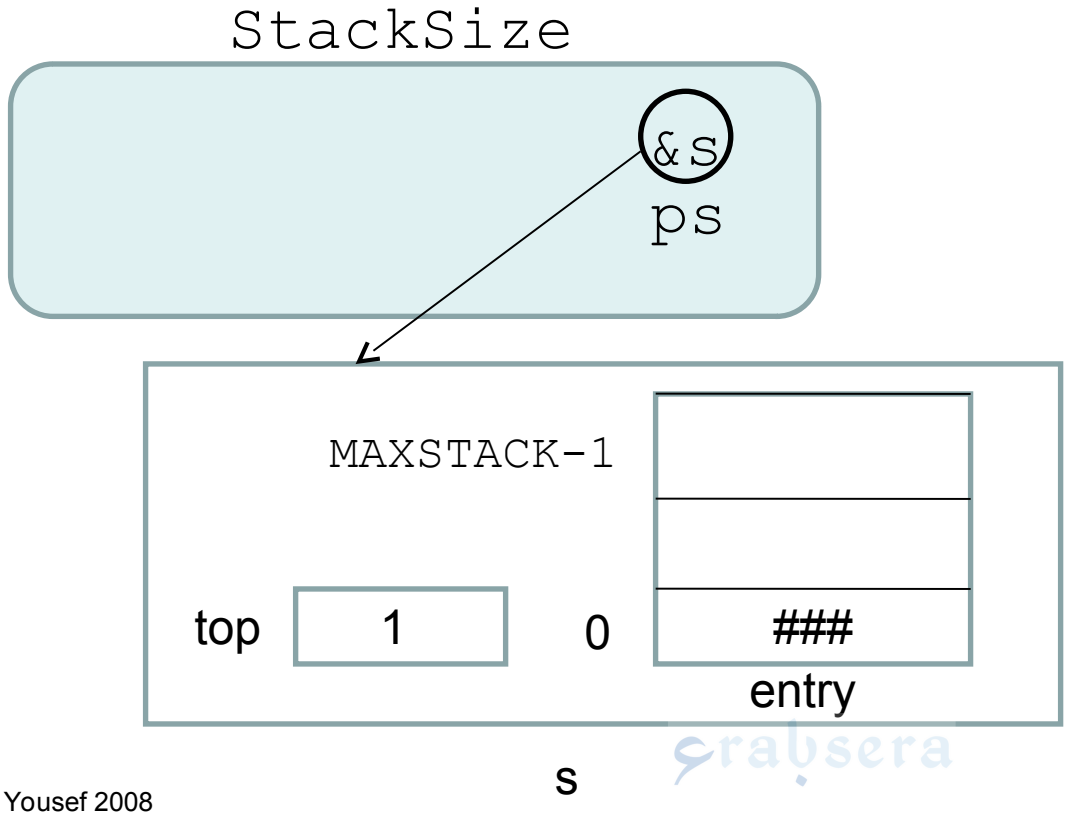
```
StackTop(&e, s)
```

but this wastes memor
and time of copying

© Waleed A. Yousef 2008

Implementation level (what really happens)

```
/*Pre: Stack is initialized.
Post: returns how many elements exist.
int StackSize(Stack *ps){
    return ps->top;
}
```



User Level (interface)

```
void main(){
StackEntry e;
Stack s;
int x;
      :
CreateStack(&s);
      :
x=StackSize(&s);
}
```

It could be:

```
StackSize(s)
```

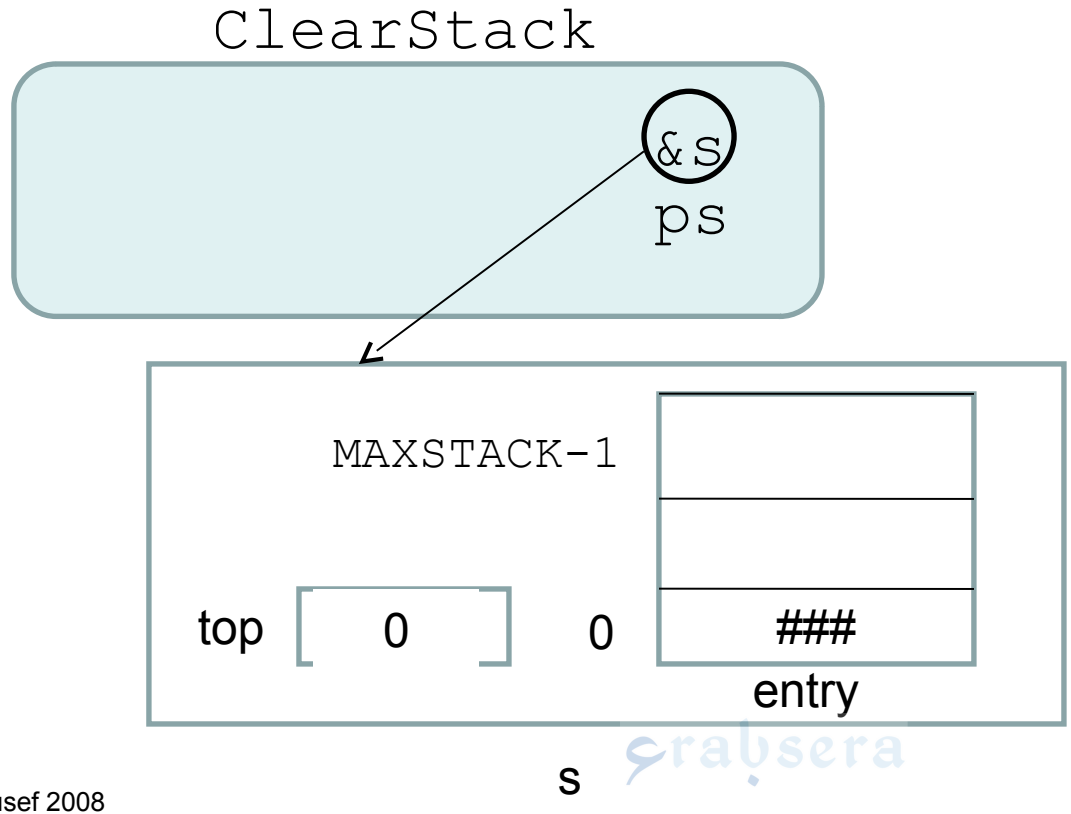
but this wastes memory and time of copying

© Waleed A. Yousef 2008

Implementation level (what really happens)

User Level (interface)

```
/*Pre: Stack is initialized.
Post: destroy all elements; stack looks initialized.
void ClearStack(Stack *ps){
   ps->top=0;
}
```




```
void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
ClearStack(&s);
}
```

Same code as `CreateStack`; why new function then?

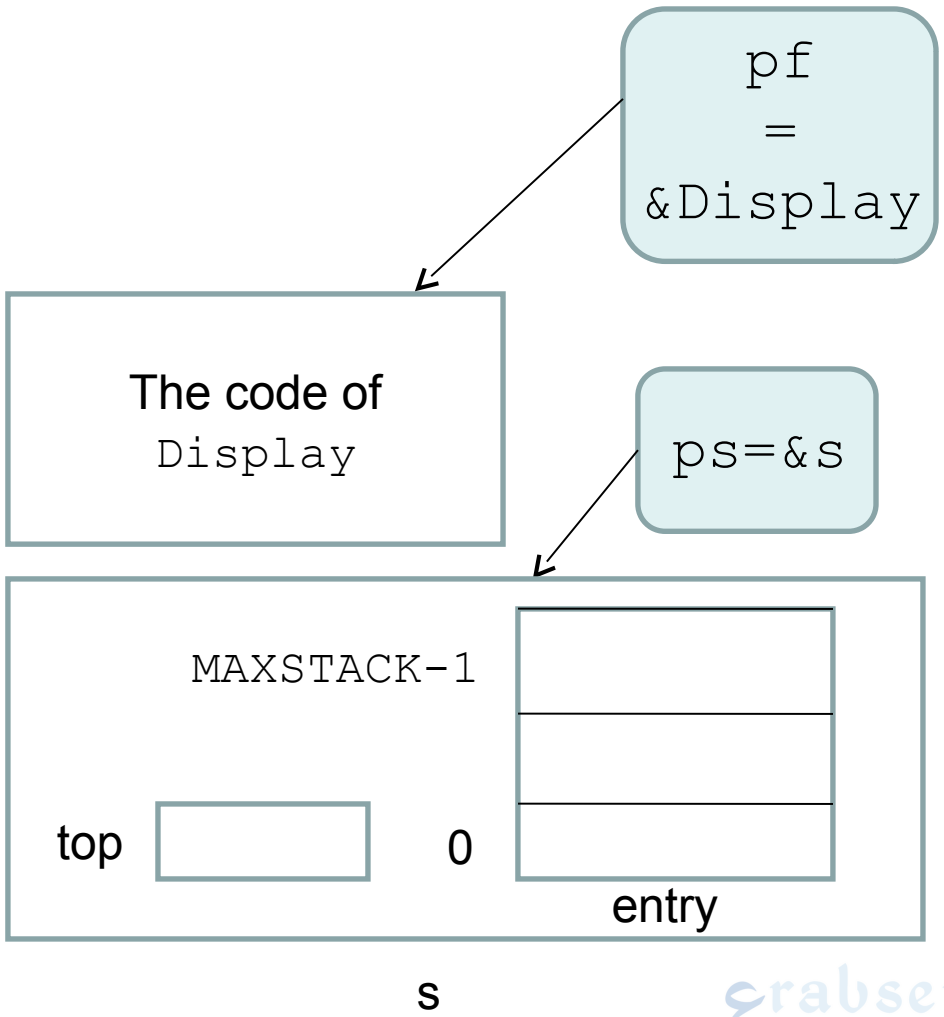
1- conceptually

2- will see later

© Waleed A. Yousef 2008

Implementation level

```
//Precondition: The stack is Initialized
void TraverseStack(Stack *ps, void(*pf)(StackEntry)){
    for(int i=ps->top; i>0; i--)
        (*pf)(ps->entry[i-1]);
}
```



User Level:

how to process each element with a user-defined function

```
void Display(StackEntry e){
   printf("e is: %d\n", e);
}

void main(){

Stack s;

CreateStack(&s);
        .
        .
TraverseStack(&s, &Display);
}
//&s only for efficiency as
said before.
```

© Waleed A. Yousef 2008

Exercise: How to write the function `StackTop` in the user level? (e.g., if you do not have the source code of the implementation)

User Level:

```
void StackTop(StackEntry *pe, Stack *ps){
   Pop(pe, ps);
   Push(*pe, ps);
}

void main(){
StackEntry e;
Stack s;
      :
CreateStack(&s);
      :
StackTop(&e, &s);
}
```

```
/* Why interface, Pre, Post are crucial:
Pop takes a pointer to the element and a
pointer to the stack.

Pre: The stack is initialized and not
empty

Post: The last element entered is returned

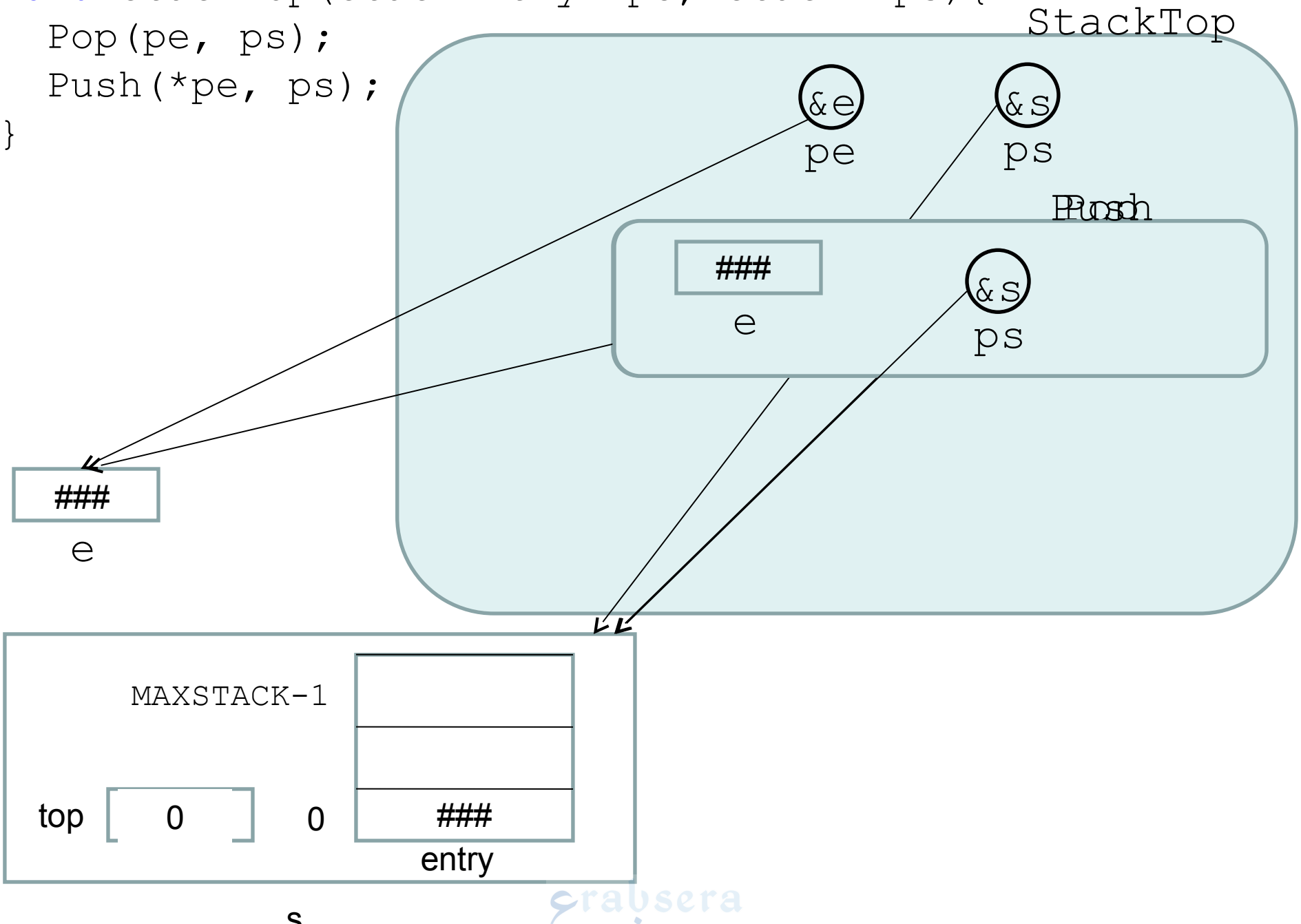
Push takes the element itself and a
pointer to the stack

Pre: The stack is initialized and not full

Post: The element e has been stored at the
top of the stack; and e does not change*/
```

© Waleed A. Yousef 2008

```
void StackTop(StackEntry *pe, Stack *ps){
   Pop(pe, ps);
   Push(*pe, ps);
}
```



© Waleed A. Yousef 2008

`StackEntry` and `MAXSTACK` should be defined in the User Level, because they concern the uer.

Also, they have to be defined in the implementation level, because they are referenced in `Stack.cpp`

Therefore, they have to be defined in `Stack.h` which is included in both `Stack.cpp` and the user main file.

So, the main file will compile even if `stack.cpp` is not compiled yet. (check the homework; this is detailed in the sheet)

© Waleed A. Yousef 2008

```
/****Stack.h*******/
#define MAXSTACK 100

typedef struct stack{
      int top;
      StackEntry entry[MAXSTACK];
} Stack;

void  Push       (StackEntry, Stack *);
void  Pop        (StackEntry *, Stack *);
int   StackEmpty (Stack *);
int   StackFull  (Stack *);
void  CreateStack(Stack *);
void  StackTop   (StackEntry *, Stack *);
int   StackSize  (Stack *);
void  ClearStack(Stack *);
void  TraverseStack(Stack *, void (*)(StackEntry));
```

© Waleed A. Yousef 2008